

Unidade II

3 PROCESSOS DE SOFTWARE

O processo de software constitui a base para o controle do gerenciamento de projetos de software e estabelece o contexto no qual são aplicados métodos técnicos, são produzidos artefatos (modelos, documentos, dados, relatórios, formulários etc.), são estabelecidos marcos, a qualidade é garantida e as mudanças são geridas de forma apropriada (Pressman, 2021, p. 15).

O processo é um conjunto de atividades, métodos, práticas e transformações empregadas no desenvolvimento e na manutenção de software. Trata-se de um guia para a equipe de desenvolvimento do software, garantindo que todos os aspectos importantes sejam explorados de maneira sistemática e controlada.

De acordo com os princípios estabelecidos em sua sexta edição, *PMBOK: guia do conhecimento em gerenciamento de projetos* (PMI, 2017), um processo de gerenciamento de projetos deve ser claramente delimitado, definindo seus pontos de iniciação e conclusão. A figura 15 ilustra essas fronteiras, representando os artefatos de entrada (inputs) e os de saída (outputs). Em projetos, esses pontos críticos são operacionalizados como marcos (milestones), elementos de referência essenciais para o monitoramento do progresso, permitindo a mensuração de desempenho, a avaliação da eficácia, a projeção de melhorias incrementais e a identificação de tendências evolutivas.

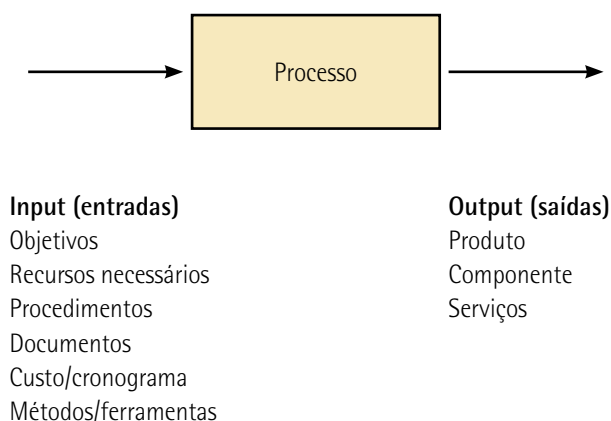


Figura 15 – Modelo básico do processo

O modelo de processo de software fornece estabilidade, controle e organização para as atividades; caso contrário, elas se tornariam caóticas.

[..] existem muitos tipos diferentes de sistemas de software, e não há um método universal de engenharia de software que seja aplicável a todos eles. Consequentemente, não há um processo de software universalmente aplicável. O processo utilizado em diferentes empresas depende do tipo de software que está sendo desenvolvido, dos requisitos do cliente de software e das habilidades das pessoas que escrevem o software (Sommerville, 2016, p. 44).

O processo ágil é uma abordagem flexível e colaborativa para desenvolver software com o foco em entregas contínuas, agregando valor ao cliente. É formado pelo conjunto de diversas outras metodologias ágeis, fundamentadas na prática para modelagem efetiva e sistemas baseados em software, e elas podem ser aplicadas por profissionais no dia a dia.



Observação

PMBOK é a sigla de Project Management Body of Knowledge (Guia do Conjunto de Conhecimentos em Gerenciamento de Projetos).

3.1 Agilidade e decomposição do processo

A agilidade no desenvolvimento de software mede a capacidade de fornecer feedbacks rápidos às mudanças no software que ocorrem ao longo de seu ciclo de vida.

Na engenharia de software, os processos são tratados de duas formas: desenvolvimento prescritivo e desenvolvimento ágil. Comumente, esses modelos são integrados para construir software. A ideia é ter um arcabouço, não só de cada processo, mas também de uma diversidade de processos que podem ser aplicados em diversas situações ou outros modelos de desenvolvimento.

O desenvolvimento prescritivo (baseado em planos) é caracterizado por seguir uma abordagem mais estruturada e sequencial, com etapas bem definidas e especificadas antes do início do projeto. Geralmente, pauta-se no modelo de processo Cascata, um clássico no desenvolvimento de software.

Observe a figura 16. São consideradas três etapas: engenharia de requisitos, especificação de requisitos e projeto e implantação. Quando ocorre uma solicitação de mudança, deve-se percorrer todo o ciclo com novas ou revisões das especificações.

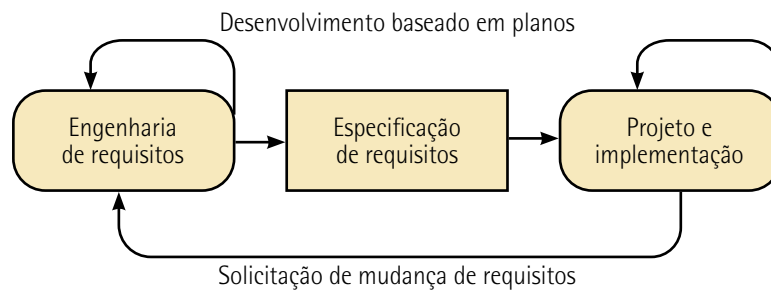


Figura 16 – Modelo prescritivo

Adaptada de: Sommerville (2016).

Esse modelo deve ser aplicado para atender toda a organização em uma visão global. Tanto o software como o processo de desenvolvimento de software é orientado por estruturas organizacionais.

O desenvolvimento prescritivo enfatiza a previsibilidade e o planejamento detalhado, enquanto o desenvolvimento ágil valoriza a flexibilidade, a colaboração e o feedback constante. Ele pode ser mais adequado para projetos estáveis e bem definidos, enquanto o modelo ágil se destaca em cenários dinâmicos e sujeitos a mudanças constantes.

O desenvolvimento ágil é uma abordagem iterativa e incremental, com baixo índice de especificação, que prioriza a adaptação a mudanças e a entrega de valor contínuo ao cliente.

Quando se utiliza o processo descritivo, basicamente, há um único modelo aplicado em todo o desenvolvimento do software, podendo inclusive conter metodologias ágeis em seu contexto. O processo ágil pode conter apenas uma das metodologias ágeis ou o conjunto dessas, conforme as necessidades do software.

Observe a figura 17. Enquanto o modelo prescritivo é formado por três etapas do desenvolvimento, em uma visão na logística de serviços, as metodologias ágeis buscam reduzir ou até zerar a etapa de especificação de requisitos.

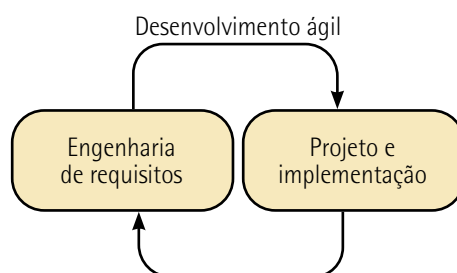


Figura 17 – Modelo de processo ágil

Adaptada de: Sommerville (2016).

A escolha de um ou mais modelos de desenvolvimento ou qualquer outro método determina os recursos alocados para construir o software, o que envolve custos e prazos.

Em um contexto geral, essa escolha depende do tamanho, da complexidade, da qualidade exigida e da tecnologia a ser implementada. Envolve também a escolha de pessoal capacitado, parceiros e uma boa comunicação.

Normalmente, um processo prescritivo é aplicado em projetos de grande escala, nos quais os requisitos são claros e estáveis.

Por sua vez, o processo ágil é aplicado em projetos pequenos, médios e em startups; denotando certo grau de incerteza. Uma das características fundamentais das metodologias ágeis é que os requisitos podem variar de acordo com novas mudanças requeridas. Mesmo que não estejam tão claros e sejam poucos estáveis, eles evoluem com o tempo por meio de feedbacks rápidos e contínuos.

Em ambos os casos, será necessário decompor os processos do projeto para avaliar o escopo e os recursos necessários para desenvolver o software.



Lembrete

O processo é um conjunto de atividades, métodos, práticas e transformações empregados no desenvolvimento e na manutenção de software.

Decompor o processo em uma sequência lógica de práticas para o desenvolvimento ou a evolução de sistemas de software engloba as atividades de análise, especificação, modelagem, implementação, testes, implantação, suporte e manutenção. Por exemplo: o levantamento de requisitos de uma funcionalidade específica.

O bom processo é caracterizado pelo emprego de dispositivos específicos de escolha do arcabouço do processo, registro e interação das ferramentas de trabalho, pessoas envolvidas no processo e métodos aplicáveis.



Observação

Funcionalidade: conjunto formado por uma ou várias funções que empregam recursos e lógicas similares, tecnologias comuns e que capacitam o software a prover funções que atendam às necessidades externas e internas do software.

Por exemplo: a funcionalidade CRUD (Create, Read, Update e Delete – Criar, Ler, Atualizar e Apagar) descreve quatro funções básicas a serem aplicadas na gestão de arquivos e operações em banco de dados.

De acordo com o escopo do projeto, o processo é decomposto, basicamente, em atividades e tarefas. Os processos podem ser subdivididos em subprocessos, a depender do escopo do projeto, e em conjunto com os procedimentos de serviços a serem seguidos. A figura 18 mostra como é a estrutura genérica de um processo e os elementos que a compõem.

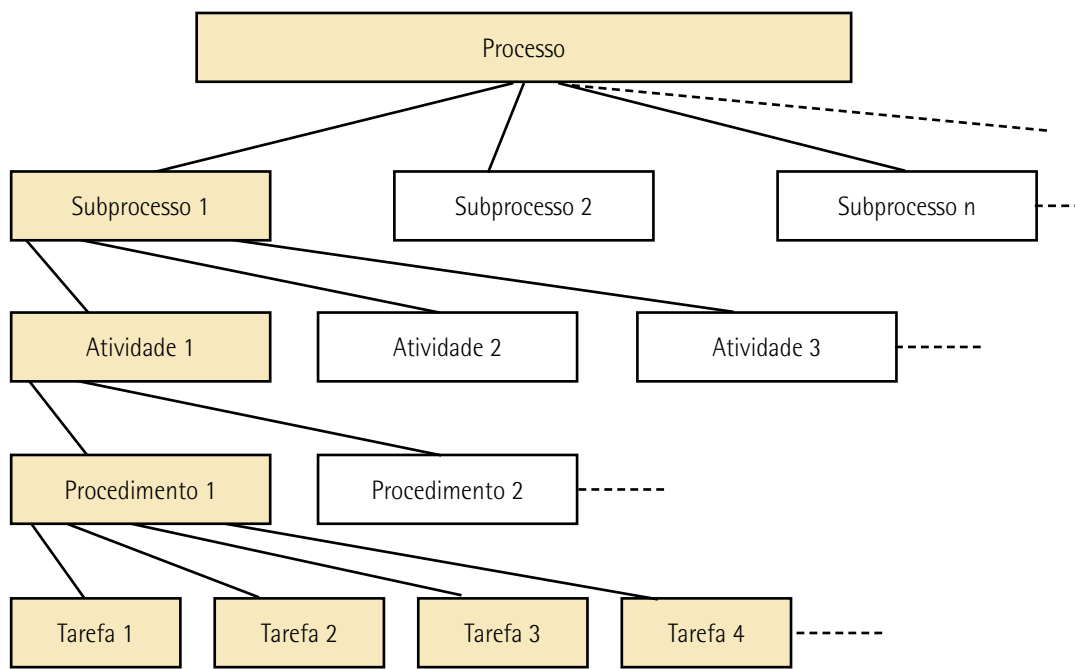


Figura 18 – Estrutura genérica do processo

Observe cada um desses elementos a seguir.

- **Subprocesso (ou meta):** é uma parte distinta de um processo maior, funcionando como se fosse uma célula de trabalho. Assim, quando se integra a outros subprocessos, contribui para o processo geral. Em cada subprocesso, nos pontos de início e fim, define-se um marco de referência, útil para o monitoramento, visando assegurar que os planos, os artefatos e as atividades de software estejam alinhados com os requisitos fixados e com o período de entrega.
- **Atividade:** é uma ação ou conjunto de ações como parte do processo ou projeto; refere-se a uma função específica ou a uma unidade de trabalho.
- **Procedimento:** é a forma pela qual se deve agir, fazer ou cumprir para que uma atividade seja realizada. Os procedimentos normalmente são caracterizados por meio de uma lista que apresenta passo a passo as tarefas a serem executadas para fazer algo de forma lógica.
- **Tarefas:** são os trabalhos que devem ser executados na atividade; representam a empreitada de serviços, ou seja, a quantidade de trabalhos realizados para efetivar a atividade.

Exemplo de aplicação

Caso: processo de implementação de uma funcionalidade.

A decomposição do processo de implementação de uma funcionalidade, apresentado na figura 19, mostra a parte referente ao subprocesso software, que corresponde a um dos itens da infraestrutura de TI, formados pelos elementos: software, hardware, pessoas, banco de dados e rede de computadores. O subprocesso "pessoas" não aparece porque nosso foco é o ambiente computacional.

Observe que o modelo mostra apenas o desenvolvimento da atividade "codificação e testes unitários". No caso, as outras atividades também devem ser decompostas porque fazem parte do subprocesso de software. Cada subprocesso também é decomposto. Esse modelo serve de guia para o processo de desenvolvimento, traz mais clareza e permite maior flexibilidade no acompanhamento do processo, podendo ser formado por guias de cartões (Kanban).

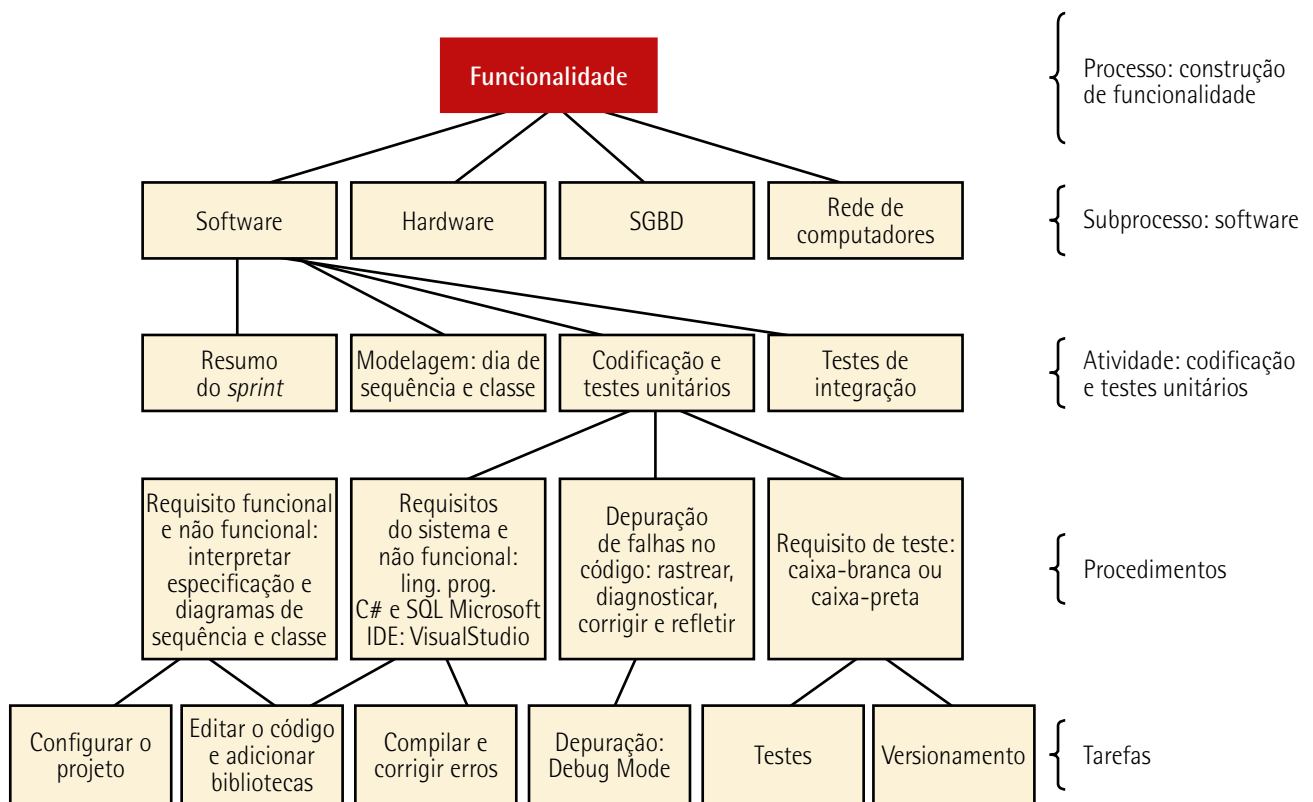


Figura 19 – Modelo para decompor o processo de implementação de funcionalidade.
Imagem criada por meio da ferramenta de diagramação draw.io



Saiba mais

O modelo apresentado na figura 19 foi construído com o draw.io. Ele é próprio para construção de arquiteturas, diagramas, protótipos, topologias e diversos outros modelos gráficos.

O draw.io tem uma coleção de funcionalidades apropriadas para a diagramação. Assim, temos diagramação, modelos e arquiteturas em um só aplicativo!

Acesse o link a seguir para aprender a usar o draw.io:

Disponível em: <https://shre.in/egu4>. Acesso em: 23 jun. 2025.

3.2 PSP (Personal Software Process – Processo de Software Pessoal)

Trata-se de uma metodologia criada pelo SEI (Software Engineering Institute). Específica para o engenheiro de software, visa melhorar a engenharia individual de software, proporcionando uma metodologia disciplinada para planejar, medir e analisar o trabalho pessoal, tentando aumentar a qualidade e a produtividade. O PSP é focado no desenvolvimento de sistemas de software por engenheiros em nível pessoal (Humphrey, 1995).

Parte integrante do CMMI (Capability Maturity Model Integration – Modelo Integrado de Capacidade e Maturidade), o PSP tem o foco na melhoria organizacional.

Desde a criação do modelo PSP, sua estrutura permanece a mesma, porém várias pequenas alterações foram feitas para torná-lo mais aplicável, eficiente e adaptável às necessidades modernas, tais como práticas como DevOps e automação de testes (Humphrey, 1995).

A proposta do PSP é interagir com as práticas organizacionais da qualidade (como ISO 9001 e ISO 9126) e modelos de qualidade, como CMMI e SPICE (Software Process Improvement & Capability Determination – Melhoria de Processos de Software e Determinação de Capacidades). O objetivo é fazer com que os processos pessoais sejam conhecidos, controlados e melhorados. Assim, o PSP:

- contribui para o cumprimento da norma ISO 9001 ao fornecer um framework de práticas individuais que são consistentes e documentáveis;
- promove a qualidade individual, o que indireta e diretamente contribui para a melhoria nas características da norma ISO 9126;
- é uma extensão pessoal que se alinha ao CMMI, pois promove a disciplina individual necessária para que as organizações alcancem níveis mais elevados de maturidade de processo;

- complementa o SPICE ao incentivar uma abordagem disciplinada e sistemática para o trabalho individual, fornecendo os dados e as métricas pessoais que podem ser usados para avaliações de capacidade de processo.

Segundo Hayes (1997), o framework do PSP, ilustrado na figura 20, apresenta sete níveis do processo e cada um se baseia no nível anterior, adicionando algumas etapas do processo a ele.

Cada nível se baseia nas capacidades desenvolvidas e nos dados históricos coletados no anterior nível. Os engenheiros aprendem a usar o PSP escrevendo dez programas, um ou dois em cada um dos sete níveis e prepara cinco relatórios escritos. Os engenheiros podem usar qualquer método de projeto ou linguagem de programação na qual eles são fluentes (Hayes, 1997, p. 6, tradução nossa).

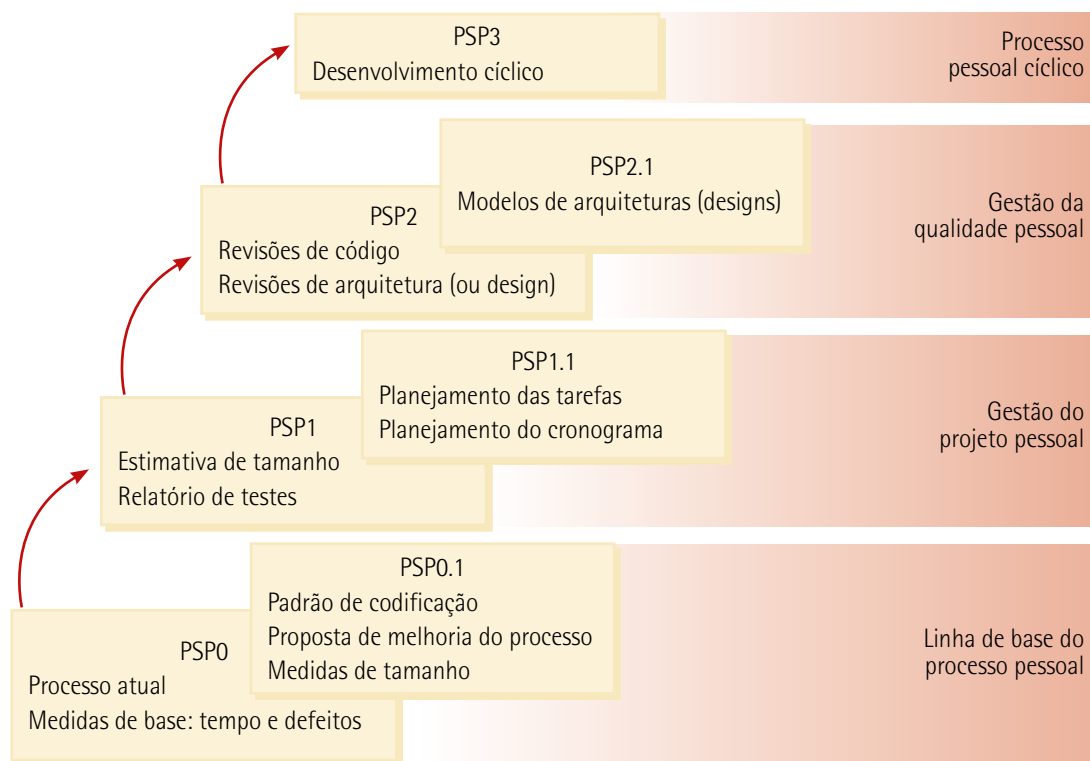


Figura 20 – Níveis do processo PSP

Adaptada de: Hayes (1997).

O PSP orienta o planejamento e o desenvolvimento dos componentes do software e tem como principais objetivos:

- **Melhoria da qualidade do software:** práticas rigorosas de revisão e inspeção reduzem o número de defeitos no software desde as primeiras fases do desenvolvimento.

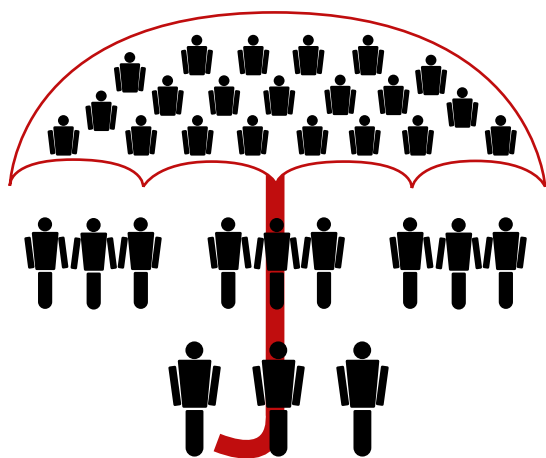
- **Precisão nas estimativas de tempo e custo:** melhora a estimativa de prazo e esforço para o desenvolvimento do software.
- **Aumento da produtividade:** um bom planejamento faz com que o índice de manutenção seja reduzido, de forma que o projeto pronto seja concluído em menos tempo, elevando a produtividade.
- **Disciplina e organização:** ajuda os desenvolvedores a seguir processos definidos e coleta de medidas de seus trabalhos, melhorando a distribuição de serviços pela equipe de desenvolvimento.
- **Análise de desempenho:** cria um comprometimento pessoal com a qualidade e a melhoria contínua do processo, fornecendo métricas para que os engenheiros avaliem e aprimorem continuamente seu desempenho com dados reais.

3.3 TSP (Team Software Process – Processo de Software da Equipe)

Trata-se de uma metodologia criada para auxiliar equipes de desenvolvimento de software a produzir produtos de alta qualidade de maneira eficiente e previsível. São práticas disciplinares que melhoram a produtividade e a capacidade na entrega do software dentro do prazo e do orçamento.

O TSP foi desenvolvido por Watts Humphrey (criador do CMMI e do PSP), com foco na equipe de trabalho. Nele, o profissional não trabalha sozinho no desenvolvimento de software.

Memorize e entenda essas três siglas na ordem em que ocorrem, conforme ilustrado na figura 21:



CMMI – melhora a capacidade da organização; foco na gestão

TSP – melhora o desempenho da equipe; foco na equipe e no produto

PSP – melhora as habilidades individuais e a disciplina; foco pessoal

Figura 21 – Métodos de melhoria de processos

Adaptada de: Humphrey (2000).

A versão inicial do CMM, v 1.0, foi revisada e utilizada pela comunidade de software durante 1991 e 1992 (Paulk, 1993, p. 3).

A partir do ano 2000, dois profissionais do CPqD (André Vilas-Boas e José Marcos Gonçalves) realizaram traduções informais do SW-CMM v 1.1 e do CMMI v 1.1, no âmbito dos projetos do Programa Brasileiro da Qualidade e Produtividade em Software, da SEPIN/MCT (Secretaria de Política de Informática – Ministério de Ciência e Tecnologia) (CMMI, 2006, p. 17).

O TSP complementa o PSP, ajudando cada membro da equipe a desenvolver e aprimorar suas habilidades pessoais dentro do contexto de trabalho em equipe, sendo que o treinamento prévio no PSP é essencial.

O TSP e o PSP podem servir como solução para pequenas organizações de software que se consideram muito pequenas para enfrentar as complexidades do CMMI.

O TSP fornece um framework para equipes de software planejar e gerenciar seus próprios processos de trabalho, acompanha o progresso e gerencia as tarefas do dia a dia.

Processos bem definidos e métricas claras fornecem à equipe um aumento significativo da qualidade e da previsibilidade de seus projetos; também integram práticas de melhoria contínua, ajudando as equipes a identificar e corrigir problemas ao longo do ciclo de vida de desenvolvimento do software.

De acordo com Pressman (2016), os objetivos para o TSP são:

- Criar equipes autogeridas que planejem e acompanhem seu próprio trabalho, estabeleçam metas e sejam proprietárias de seus processos e planos. As equipes podem ser puras ou equipes de produto integradas (IPTs – Integrated Product Teams), com cerca de 3 a 20 engenheiros.
- Mostrar aos gerentes como treinar e motivar suas equipes e como ajudá-las a manter alto desempenho.
- Acelerar o aperfeiçoamento dos processos de software, tornando o comportamento CMM nível 5 algo normal e esperado.
- Fornecer orientação para melhorias a organizações com elevado grau de maturidade.
- Facilitar o ensino universitário de habilidades de trabalho em equipe de nível industrial.



Observação

CMM nível 5 é conhecido como o nível de otimização, o nível mais alto de maturidade descrito no modelo.

O TSP oferece um conjunto de elementos fundamentais para ajudar as equipes de desenvolvimento a entregar software de alta qualidade de forma previsível e eficiente. Observe esses aspectos no quadro a seguir.

Quadro 3

Elementos fundamentais	
Papéis e responsabilidades	Os papéis são definidos para cada membro da equipe, podendo adquirir as seguintes funções como papéis: responsável pela interação com o cliente, responsável pelo design, responsável pela implementação, gestor de testes, gestor de planejamento, gestor de processos, gestor de qualidade, responsável pelo suporte e líder de equipe
Planejamento detalhado	Envolve a criação de planos, que incluem cronograma, alocação de recursos e marcos de referência
Ciclos de desenvolvimento	Trata-se da divisão das tarefas em ciclos curtos e iterativos para o desenvolvimento de determinadas funcionalidades
Garantia da qualidade	Engloba a prática disciplinada de controle de qualidade, revisões de código, inspeções e testes que garantem que o software atenda aos padrões de qualidade
Métricas e monitoramento	Métricas de desempenho aplicáveis para a coleta e a análise contínua de dados são usadas para monitorar o progresso do projeto e tomar decisões informadas
Comunicação e coordenação	São empregadas ferramentas e práticas que favorecem uma comunicação eficaz e a coordenação entre os membros da equipe
Gerenciamento de riscos	Envolve a identificação e a mitigação de riscos potenciais que podem afetar o desenvolvimento do projeto
Incidentes e defeitos	O registro e o acompanhamento de defeitos e incidentes encontrados ao longo do ciclo de desenvolvimento têm o objetivo de prever e resolver problemas de forma rápida
Revisão e melhoria contínua (iteração)	A reflexão dos ciclos de desenvolvimento ajuda a identificar áreas de melhoria e implementar mudanças no processo
Documentação	Trata-se do registro detalhado do projeto, incluindo planos, métricas, resultados de testes e evidências de defeitos



Lembrete

O TSP é uma metodologia criada para auxiliar equipes de desenvolvimento de software a produzir produtos de alta qualidade de maneira eficiente e previsível.

Esse modelo básico de metodologia do processo de software permite que as organizações se orientem por um modelo de processo, que serve de base para estruturar a organização do desenvolvimento.

Uma metodologia genérica do processo para a engenharia de software estabelece cinco atividades metodológicas: comunicação, planejamento, modelagem, construção e entrega. Além disso, um conjunto de atividades de apoio que é aplicado ao longo do processo, como o acompanhamento e o controle do projeto, a administração de riscos a garantia da qualidade, o gerenciamento das configurações, as revisões técnicas, entre outras (Pressman, 2021, p. 18).

Os modelos de processos de software englobam um conjunto de atividades, métodos, práticas e transformações empregadas no desenvolvimento e na manutenção de software. Eles fornecem estabilidade, controle e organização para as atividades, que, sem controle, tornam-se bastante caóticas.

Para o projeto ou partes específicas dele, a adoção de um modelo de processo de software permite ter uma visão geral do sistema como um processo orientado por uma estrutura organizacional.

Os modelos de processo de software são mostrados a seguir:

- **Modelos de processos:** PSP e TSP.
- **Modelos de processos tradicionais:** Cascata, Balbúrdia, Prototipagem, Incremental, RAD (Rapid Application Development – Desenvolvimento Rápido de Aplicação) e Espiral.
- **Processo unificado:** RUP, Praxis e Iconix.

3.4.1 Cascata (Waterfall Model ou Sequencial Linear)

O modelo Cascata, algumas vezes chamado ciclo de vida clássico, sugere uma abordagem sequencial e sistemática para o desenvolvimento de software, começando com a especificação dos requisitos do cliente, avançando pelas fases de planejamento, modelagem, construção e entrega, e culminando no suporte contínuo do software concluído (Pressman, 2016, p. 64).

Ilustrado na figura 23, ele é considerado o modelo clássico do ciclo de vida de desenvolvimento do software, pois provavelmente foi o primeiro a ser apresentado.

Como observado na figura 23, o processo só termina após a última atividade, a entrega feita ao cliente, só depois o software passa por revisões que possibilitam alguma mudança.

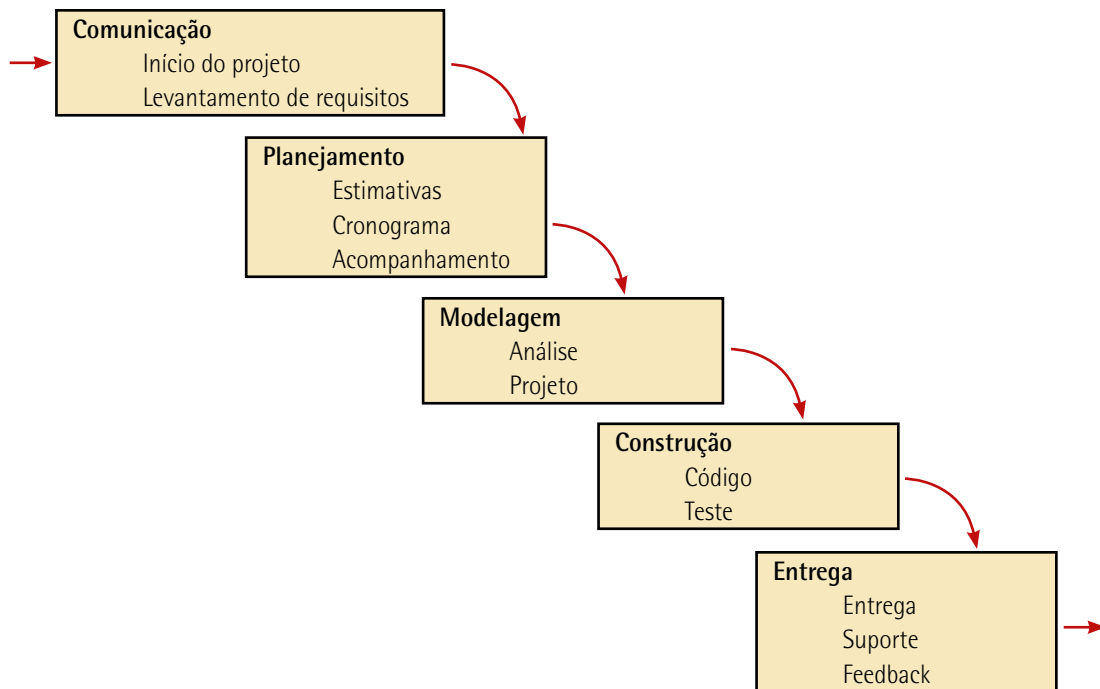


Figura 23 – Cascata

Adaptada de: Pressman (2021).

São descritos a seguir os conceitos sobre as atividades do Cascata:

- **Comunicação**: é a fase de elicitación, quando se inicia o projeto. Nela, é feito o levantamento de requisitos, determinando as funcionalidades, as restrições e os objetivos do sistema.
- **Planejamento**: nessa atividade, os requisitos são agrupados para definir o escopo do sistema, determinar a arquitetura modelada, estimar prazos e custos. São produzidos artefatos de cronogramas, análises de caminho crítico e seleção de recursos a serem disponibilizados. Com os artefatos construídos, é feito o acompanhamento de cada etapa do projeto.
- **Modelagem**: toda modelagem é acompanhada de sua especificação, além do processo de modelagem do negócio, que está na atividade "Comunicação". Os modelos a serem criados envolvem três arquiteturas: modelagem da informação – diagrama de classes e de pacotes; modelagem da lógica de processamento – diagramas de UC, sequência, estado e outros; e modelagem da infraestrutura de TI – diagrama de componentes e de implantação.
- **Construção**: refere-se à codificação e aos testes a serem realizados do componente de software produzido. A verificação do requisito ocorre pelo atendimento das especificações, pela inspeção de código e pelos testes unitários e de integração. As unidades são integradas e testadas como um sistema completo.

- **Entrega:** é a implantação do software no ambiente do cliente. Com o suporte necessário, feedbacks são colhidos de acordo com as observações dos usuários, tais como erros que não foram identificados anteriormente, melhoria na interface, aumento de funções para o sistema e descoberta de novos requisitos.

Embora o Cascata seja frequentemente utilizado como base para outros paradigmas de desenvolvimento, apresenta vantagens específicas em cenários que envolvem sistemas estruturados, caracterizados por requisitos bem definidos, ambientes operacionais restritos e de alta capacidade, além de aplicações que demandam a integração de múltiplos componentes em soluções complexas. Sua utilização é mais apropriada para problemas complexos ou de difícil resolução, quando o planejamento detalhado e sequencial facilita a coordenação das etapas.

Deve-se acentuar que o Cascata apresenta limitações consideráveis, especialmente devido à sua inflexibilidade na segmentação do projeto em fases distintas e sequenciais. Na prática, raramente é possível capturar todos os RS de maneira completa e precisa no início do projeto. Além disso, problemas identificados em etapas anteriores só são percebidos após a entrega do produto final, resultando em retrabalhos substanciais ou, em casos extremos, na inviabilidade do projeto. Outro ponto crítico é que ele adota uma perspectiva de alto nível que não reflete com precisão o processo real de desenvolvimento de software. Alterações nos requisitos ou no escopo durante o desenvolvimento são significativamente desestimuladas, tornando-o inadequado para projetos que usam paradigmas mais dinâmicos, como o desenvolvimento orientado a objetos.

3.4.2 Balbúrdia (codifica/corrigir ou codifica/remenda)

Ilustrado na figura 24, é uma abordagem de desenvolvimento de software caracterizada pela ausência de planejamento formal, estrutura organizada ou definições claras dos requisitos.

Nesse modelo, o desenvolvimento é conduzido de forma improvisada e iterativa, com foco em resolver problemas imediatos ou entregar funcionalidades rapidamente, sem considerar uma visão de longo prazo ou práticas consolidadas de engenharia de software. Existe um ciclo contínuo de mudanças não planejadas que distancia cada vez mais o software de seu estado de maturidade e estabilidade.

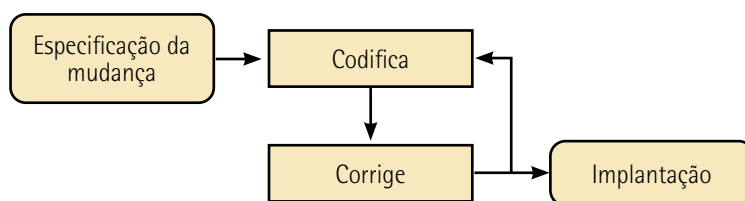


Figura 24 – Balbúrdia

Logicamente, o Balbúrdia não é um modelo a ser seguido para desenvolver software, porém inúmeros profissionais o seguem, mesmo não sabendo disso. Ele é caracterizado pela administração de crises, informalidade, loop de gestão codifica/corrigir, em que o planejamento, os requisitos do software, a modelagem e a documentação são caóticos ou até mesmo inexistentes.

3.4.3 Prototipagem

Protótipos são produtos em menor escala, modelos 2D e 3D gerados em computador ou simulações. Os protótipos permitem que as partes interessadas façam experiências com um modelo do seu produto final, em vez de somente discutirem representações abstratas dos requisitos. Os protótipos suportam o conceito de elaboração progressiva em ciclos iterativos de criação de modelos, experimentação de usuário, geração de feedbacks e revisão do protótipo. Quando ciclos de feedback suficientes forem realizados, os requisitos obtidos a partir do protótipo estarão completos o suficiente para passar para a fase de design ou de construção (PMI, 2017, p. 147).

A prototipagem tem como objetivo principal simular o comportamento e a aparência do sistema final. O projeto passa por várias investigações para garantir que o desenvolvedor, o usuário e o cliente cheguem a um consenso sobre o que é necessário e o que deve ser proposto.

Antes da entrega definitiva, são desenvolvidos protótipos que servem como representações iniciais ou intermediárias do sistema. Eles permitem identificar e analisar potenciais problemas, além de facilitar a comunicação e o alinhamento entre desenvolvedores e clientes, promovendo um entendimento mais claro dos requisitos e das expectativas do projeto.

O protótipo de software é um mecanismo para identificar requisitos de IHC, RF, práticas de UX em mapas de navegação interativos, mapa de estilos para CSS e IDE (Integrated Development Environment – Ambiente de Desenvolvimento Integrado).

As atividades da prototipagem podem ser determinadas a partir do paradigma de prototipagem, conforme ilustrado na figura 25:

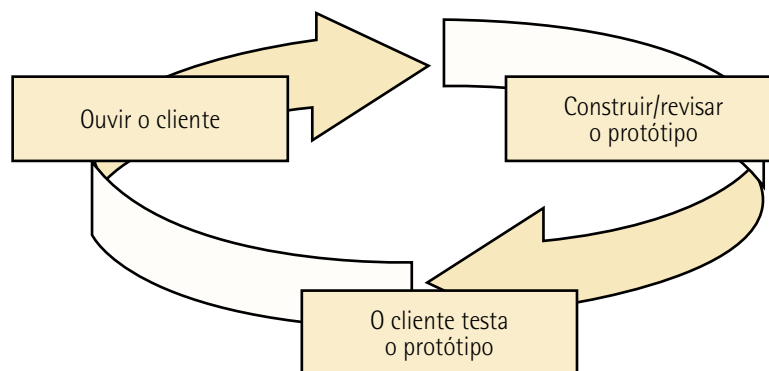


Figura 25 – O paradigma de prototipagem

Adaptada de: Pressman (2011).

Em uma análise do paradigma de prototipagem, o ciclo de atividades da prototipagem tem os seguintes estágios:

1) Começa com um conjunto simples de requisitos fornecido pelos clientes e usuários para estabelecer objetivos do protótipo. Nesse instante, é preciso ter o bom senso de selecionar apenas o que é relevante como requisito.

2) O protótipo é construído, podendo ser apenas um esboço ou algum modelo gráfico desenhado com uma ferramenta de software.

3) A revisão permite que clientes e usuários façam testes e experimentações, assim, eles decidem o que querem. Depois, os requisitos são revisados, alterados, detalhados, documentados e o sistema passa a ser codificado.

4) O estágio final do processo, como descreve Sommerville (2016), é a avaliação do protótipo. Durante esse estágio, provisões devem ser feitas para o treinamento do usuário e os objetivos do protótipo devem ser usados para derivar um plano de avaliação. Os usuários necessitam de um tempo para se sentirem confortáveis com um sistema novo e se situarem em um padrão normal de uso. Ao usar o sistema normalmente, eles descobrem erros e omissões de requisitos.

As atividades da prototipagem podem ser classificadas de acordo com a fase de prototipagem do software:

- planejamento de definição dos requisitos;
- construção de esboço (croqui);
- wireframe;
- protótipos de diversos tipos com baixo ou alto grau de detalhes;
- testes de usabilidade e feedback;
- interação;
- refinamento;
- protótipo final.

Devido à falta de familiaridade dos usuários com aspectos técnicos avançados, esse método de desenvolvimento é particularmente eficaz, permitindo uma participação significativa dos usuários no processo de interação com a solução proposta.

Entre as vantagens da prototipagem, destacam-se:

- **Ciclo de vida eficiente:** permite ao desenvolvedor criar um modelo representativo do software a ser construído.
- **Adequação a objetivos definidos:** é apropriada quando o cliente definiu os objetivos gerais do software, mas ainda não especificou ou identificou detalhes dos requisitos de entrada, processamento e saída.
- **Identificação antecipada de requisitos:** facilita a análise conjunta entre desenvolvedor, cliente e usuário; mitigando riscos e incertezas durante o desenvolvimento. O crucial é definir as regras do jogo logo no começo.
- **Aceleração do desenvolvimento:** proporciona uma visão mais tangível do software em desenvolvimento para o usuário, permitindo a visualização antecipada de telas e relatórios resultantes.
- **Envolvimento direto do usuário:** transformando-o em um coautor do projeto, o que aumenta a precisão das especificações e melhora a usabilidade do produto final.

Como desvantagens, acentuam-se:

- **Percepção do cliente sobre a totalidade do software:** quando o cliente não está ciente de que o software em teste é apenas um protótipo, não considera a qualidade global ou a manutenibilidade necessária durante a operação do software, isso pode levar a expectativas mal alinhadas. O simples fato de o usuário clicar em um botão do protótipo pode fazê-lo questionar, por exemplo, que o botão não está "chamando" o algoritmo da função. O algoritmo pode ainda nem existir, isto é, o que está sendo apresentado é apenas um layout de tela.
- **Expectativas do usuário:** a prototipagem pode criar a falsa impressão de que todas as sugestões dos usuários podem ser facilmente implementadas, independentemente da fase do desenvolvimento. Além disso, os usuários podem não entender o motivo da demora na entrega final após visualizar uma versão demo, levando a frustrações e pressões para que o protótipo seja utilizado como produto final.

Um dos exemplos de protótipos de tela pode ser visto na figura 26. Ela mostra o resultado da prática de diagramação (layout ou wireframe) de tela, que é um protótipo que mostra espaços reservados para os signos de imagem: caixas de texto, caixas de imagem, botões, barras de rolagem, enfim, todos os elementos necessários de IHC.

Em portais e diversos sites, é comum ter um ou mais modelos de telas diagramados. Por vezes, esses espaços são vendidos, reservados por empresas para publicidade, propaganda, notícias e outras aplicações.

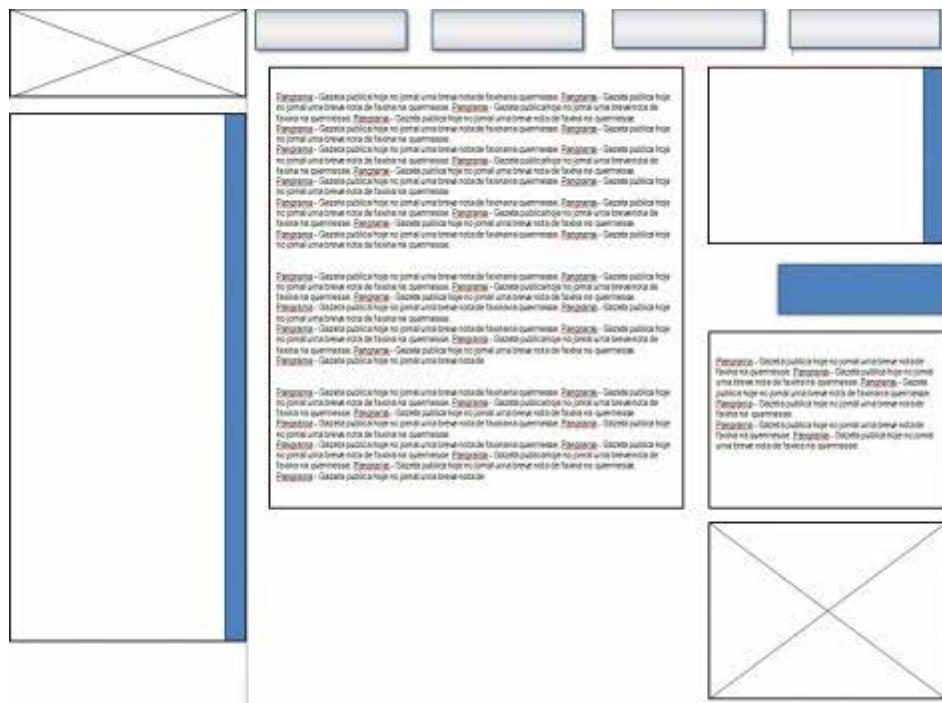


Figura 26 – Diagramação de tela web



Saiba mais

A engenharia de usabilidade se refere a conceitos e técnicas para análise, síntese e avaliação de usabilidade em sistemas (Preece, 2002 *apud* Martins, 2011).

A usabilidade a ser avaliada no projeto diz respeito à relação do software com as tarefas dos usuários e com o objetivo de confirmar o nível em que os requisitos da organização e dos usuários foram alcançados, fornecendo também informações para o refinamento do projeto.

Para entender melhor como elaborar um teste de usabilidade, consulte o site de Neil Patel.

Disponível em: <https://shre.ink/egup>. Acesso em: 23 jun. 2025.



Figura 27 – Design de IU

Disponível em: <https://shre.ink/xEeE>. Acesso em: 23 jun. 2025.

3.4.4 Incremental

É projetado para atender as exigências empresariais e usuais de prazos acelerados e alta qualidade. Ilustrado na figura 28, esse modelo operacionaliza a liberação do sistema em partes incrementais, cada uma delas entregando funcionalidades utilizáveis enquanto o desenvolvimento contínuo prossegue em paralelo.

Esse modelo incorpora elementos sequenciais do Cascata de forma evolucionária. Ele permite a entrega de incrementos iterativos com base em marcos de entrega, aprovação e validação, assegurando que o usuário utilize partes do sistema em desenvolvimento desde cedo, minimizando o tempo de espera total e adaptando-se às necessidades emergentes ao longo do ciclo de desenvolvimento.

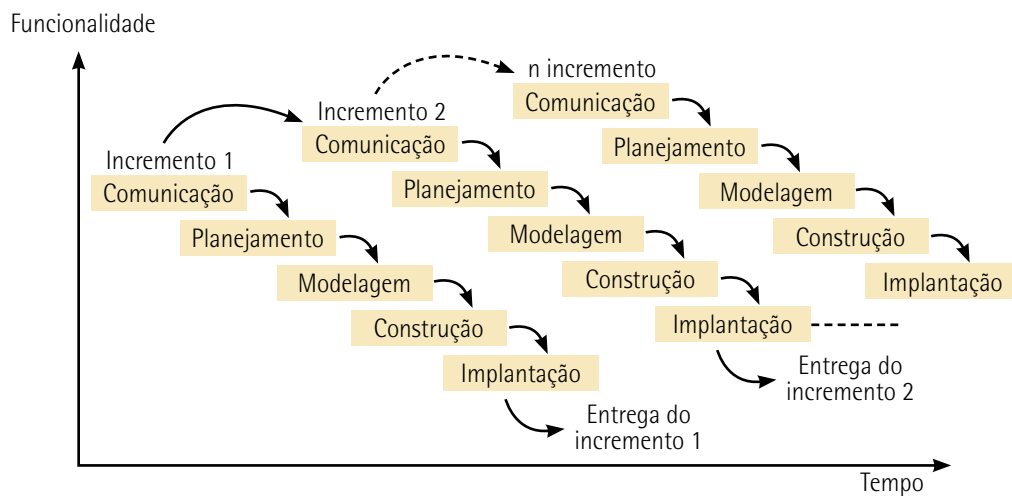


Figura 28 – Modelo de processo incremental

Adaptada de: Pressman (2007).

O processo incremental é caracterizado por sua natureza iterativa, com o objetivo de apresentar um produto operacional com cada incremento. Esse método é fundamentado no conceito de versões, quando iterações sequenciais são utilizadas para aprimorar continuamente o software.



Observação

Iteração é uma estratégia de planejamento que envolve retrabalho no processo de desenvolvimento, revisões de prazos, identificação e correção de falhas, erros e melhorias tecnológicas. Esse ciclo contínuo visa melhorar o sistema e distribuir as tarefas de forma eficiente.

A cada iteração, novas versões são geradas, registrando as mudanças implementadas no software até a entrega do release, que é a versão finalizada para os usuários. As iterações do processo são necessárias para a melhoria contínua do processo de software.

Nesse modelo, o sistema é especificado na documentação dos requisitos e decomposto em subsistemas funcionais. As versões são inicialmente definidas a partir de um pequeno subsistema funcional, com novas funcionalidades sendo adicionadas a cada versão seguinte. O software alcança sua funcionalidade total gradualmente por meio desses incrementos sucessivos.

3.4.5 RAD

Outros modelos de desenvolvimento apresentam grande utilidade, muitos dos quais resultam da combinação de conceitos de diferentes metodologias, como é o RAD. Trata-se de uma abordagem mista, com características genéricas, sendo amplamente utilizada no desenvolvimento de software.

O RAD permite que uma equipe de desenvolvimento crie um sistema totalmente funcional em um período de tempo reduzido. Conforme mencionado por Pressman (2002), o modelo RAD é um processo de software incremental que enfatiza um ciclo de desenvolvimento curto.

É uma abordagem que visa à entrega rápida de software, frequentemente utilizando ferramentas especializadas de programação de bancos de dados e apoio ao desenvolvimento, como geradores de interfaces e relatórios (Sommerville, 2011).

O RAD adota uma estratégia baseada na construção modular e orientada a componentes reutilizáveis. Ele permite acelerar o desenvolvimento de sistemas, reduzindo significativamente o tempo necessário para a entrega de soluções de complexidade moderada.

Conforme ilustrado na figura 29, é possível concluir o desenvolvimento completo de um sistema nesse modelo em um intervalo de 60 a 90 dias, dependendo da complexidade dos requisitos e da integração eficiente dos componentes.

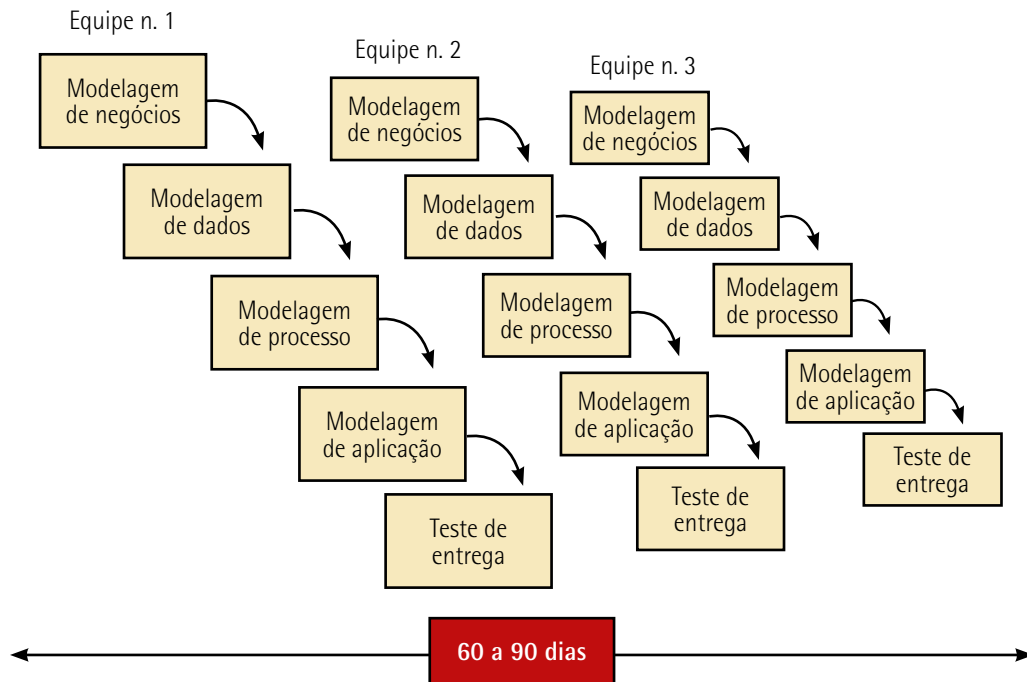


Figura 29 – Modelo RAD

Adaptada de: Pressman (2007).

As principais vantagens desse modelo são:

- **Ciclo de desenvolvimento curto:** segue uma abordagem sequencial linear com ciclos de desenvolvimento significativamente reduzidos, permitindo entregas em intervalos curtos, geralmente entre 60 e 90 dias.
- **Modularidade e componentização:** a construção baseada em componentes reutilizáveis permite modularizar o sistema, facilitando o gerenciamento do projeto e a identificação precoce de desvios no escopo.
- **Requisitos claramente delimitados:** os requisitos iniciais devem ser suficientes para viabilizar o escopo do projeto, otimizando o foco da equipe.
- **Escalabilidade para sistemas grandes:** o modelo é ideal para aplicações de sistemas de informação extensos e modularizados, possibilitando que funções ou módulos principais sejam atribuídos a equipes independentes e que depois integram os componentes ao sistema completo.
- **Visibilidade incremental:** a entrega contínua de incrementos oferece maior visibilidade ao cliente, que pode acompanhar e validar partes do sistema ao longo do desenvolvimento.

Entre suas principais desvantagens, acentuam-se:

- **Dependência de modularidade:** o sucesso do RAD depende da adequada modularização do sistema. Caso a segmentação dos componentes seja ineficiente, o desenvolvimento pode enfrentar dificuldades significativas.
- **Inadequação para riscos técnicos elevados:** o modelo pode não ser apropriado quando o projeto envolve tecnologias emergentes ou pouco dominadas pela equipe, aumentando os riscos técnicos.
- **Necessidade de recursos humanos intensivos:** a abordagem exige equipes dedicadas para cada módulo ou funcionalidade, o que pode ser inviável em projetos com limitações de pessoal.
- **Comprometimento intenso:** tanto desenvolvedores quanto clientes devem estar altamente comprometidos com as atividades de ritmo acelerado ("fogo-rápido") para garantir que os prazos curtos sejam cumpridos.
- **Limitações em aplicações específicas:** nem todos os tipos de sistemas ou aplicações se adaptam bem ao modelo RAD. Projetos que requerem alta integração ou sincronização entre componentes podem enfrentar dificuldades com essa abordagem.
- **Dependência de sincronismo e desempenho:** quando o desempenho do sistema está fortemente vinculado ao sincronismo das interfaces entre componentes, o RAD pode não ser eficiente, comprometendo o resultado final.

3.4.6 Espiral

Conforme ilustrado na figura 30, é uma abordagem evolucionária que integra a natureza iterativa da prototipagem com a estrutura sequencial do Cascata. O desenvolvimento do software ocorre por meio de múltiplas versões incrementais, evoluindo progressivamente a partir de ciclos iterativos.

Em cada iteração da espiral, são realizadas atividades estruturais previamente definidas no arcabouço, abrangendo desde o planejamento e a análise de riscos até a implementação e a avaliação.

Ao final de cada ciclo, uma versão parcial ou completa do sistema é entregue (release), com novas funcionalidades ou melhorias incorporadas. Após múltiplas iterações, o software atinge sua totalidade funcional, contemplando todos os requisitos estabelecidos.

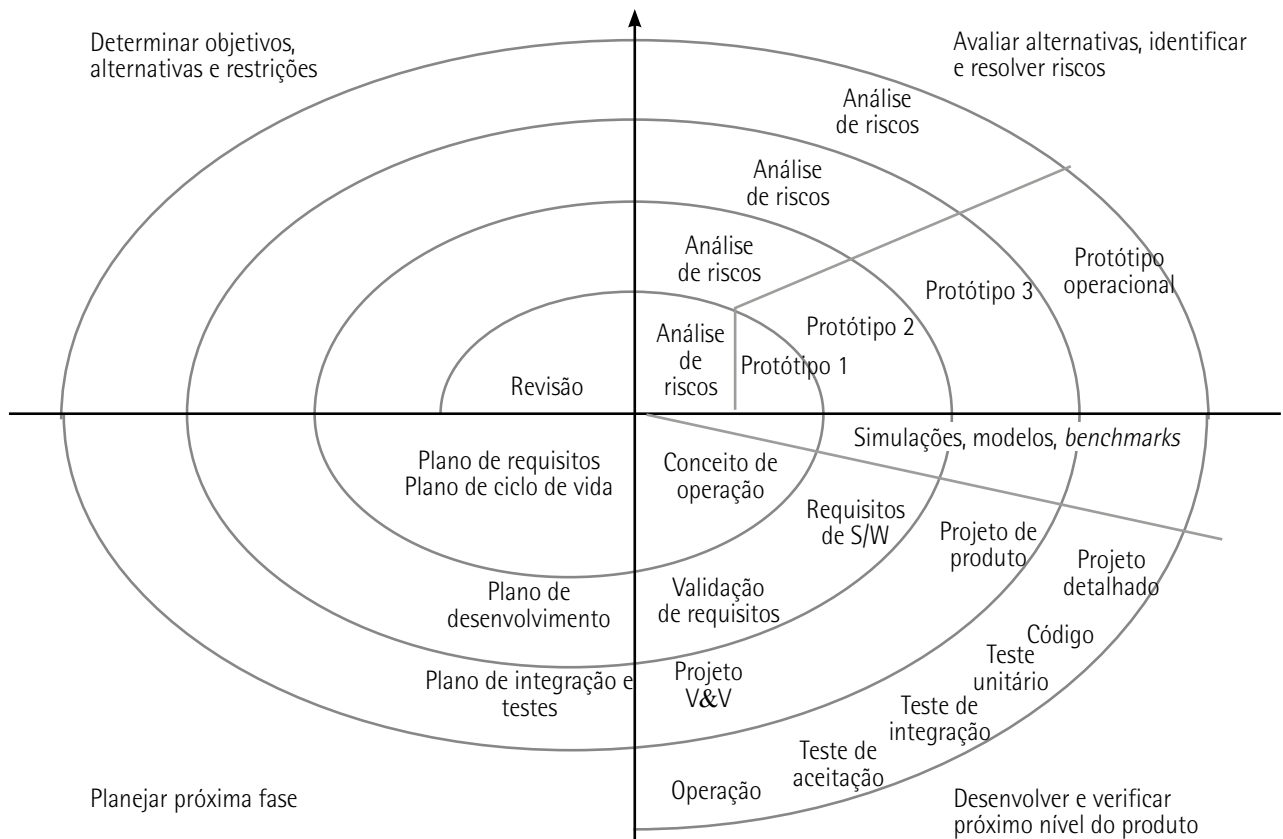


Figura 30 – Modelo de processo espiral

Fonte: Sommerville (2016, p. 33).

O modelo Espiral é adequado para sistemas complexos, projetos de software de grande porte e aqueles que demandam um alto nível de interação com os usuários. Sua abordagem iterativa permite identificar e resolver problemas progressivamente, garantindo maior alinhamento com os requisitos e os objetivos do sistema.

O modelo é dividido em quatro etapas principais, que se repetem ao longo de cada ciclo da espiral:

- **Ativação:** definem-se os objetivos específicos, identificam-se as restrições para o processo e é preparado um plano de gerenciamento detalhado. Identificam-se também os riscos sem analisá-los profundamente (foco da próxima fase).
- **Análise de riscos e prototipagem:** com base nos riscos identificados na fase anterior, são realizadas análises detalhadas e tomadas providências para amenizar esses riscos. Criam-se várias versões de protótipos para apoiar essa fase.
- **Desenvolvimento:** fundamentado pelas fases anteriores, escolhe-se o modelo mais adequado para o desenvolvimento do software. A experiência profissional e a vivência do desenvolvedor em outros sistemas são estratégias para essa fase. Dependendo da complexidade ou do tamanho do sistema, por vezes, é necessária a presença de um consultor especialista.

- **Planejamento:** o projeto é revisto nessa fase e é tomada uma decisão de realizar um novo ciclo, na espiral ou não. Se continuar com o aperfeiçoamento do sistema, é traçado um plano para a próxima fase do projeto. Um diferencial nesse modelo comparado com outros é a explícita consideração de riscos dentro do projeto como um todo. Para tanto, criou-se uma fase específica de análise de riscos nesse modelo.

4 PLANEJAMENTO DO PROCESSO DE SOFTWARE

Refere-se à definição detalhada das atividades, metodologias e práticas a serem adotadas ao longo do ciclo de vida do desenvolvimento de software, com o objetivo de garantir a entrega de um produto de alta qualidade, dentro dos requisitos de tempo e custo estipulados.

Cada modelo de processo representa o processo em uma perspectiva particular, por exemplo, o Incremental é acompanhado do versionamento, que acompanha a evolução ou as mudanças do software por meio de registros de entrega, como uma funcionalidade. No caso Incremental, são apresentadas as atividades sequenciais desenvolvidas ao longo de seu ciclo; no entanto, não mostra a definição dos papéis e das responsabilidades das pessoas envolvidas no projeto.

De acordo com Sommerville (2016), um modelo de processo de software é uma representação simplificada de suas atividades, que acompanham todo o SDLC (Software Development Life Cycle – Ciclo de Vida de Desenvolvimento de Software).

O planejamento relacionado ao processo de desenvolvimento acompanha cada etapa do SDLC, o que inclui abordagens técnicas na escolha dos seguintes aspectos:

- modelos de processo de desenvolvimento;
- definição de metodologias e frameworks de como as atividades serão executadas;
- planejamento de iterações e releases;
- gestão da qualidade e testes;
- gestão e alocação de recursos;
- gestão do risco;
- definição das ferramentas que automatizam cada uma das atividades ou tarefas a serem executadas.

4.1 SDLC

Essencialmente, ele é adaptado para uma estrutura organizacional específica, ilustrada na figura 31. Esta detalha as diferentes fases da organização, que devem ser alinhadas a um ciclo de vida, e as inter-relações entre elas, que afeta diretamente como cada fase do ciclo é planejada, executada e monitorada.

O alinhamento do SDLC com estruturas organizacionais envolve as atividades, as funções (papéis) das pessoas, as responsabilidades e os fluxos de trabalho dentro da organização do processo de desenvolvimento.

A estrutura influencia o grau de autoridade e de responsabilidades, que podem ser centralizadas ou distribuídas, podendo ter uma estrutura horizontal ou operar com equipes de projeto auto-organizáveis, como é o caso das metodologias ágeis.

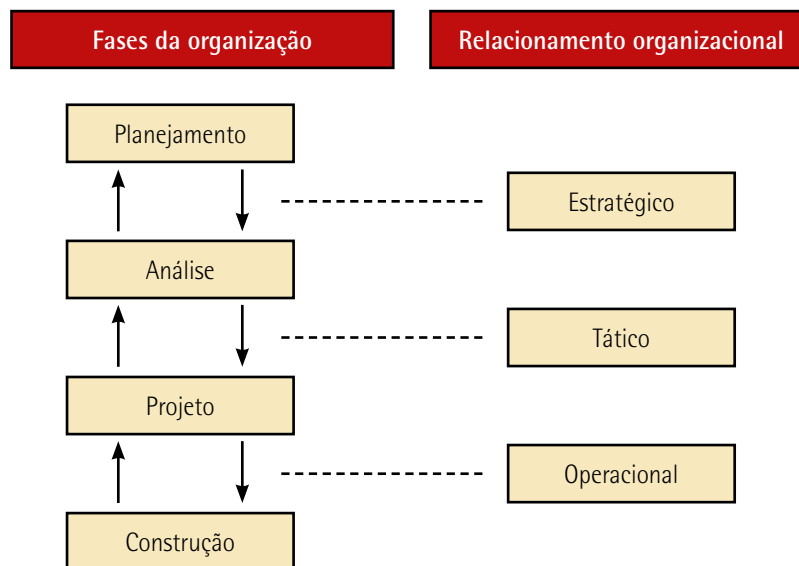


Figura 31 – Estrutura para o desenvolvimento de software com suas fases e formas de relacionamento

O ciclo de vida do processo de software, acompanhado por um cronograma, envolve planejar, monitorar e controlar o tempo dedicado a cada fase do ciclo de vida.

O cronograma é uma ferramenta prática para gerenciar o tempo e as entregas do projeto de software, enquanto o ciclo de vida do processo define as etapas estruturadas pelas quais o desenvolvimento do software passa. A integração de ambos visa garantir que o software seja entregue no tempo previsto, atendendo aos requisitos e padrões de qualidade.

No planejamento do processo de software, cada fase da estrutura organizacional é acompanhada de seus processos e cada um deverá ser executado com base em métodos, ferramentas e técnicas, que fazem parte das tarefas da equipe de desenvolvimento. O quadro 4 exemplifica algumas das escolhas de MF&T.

Quadro 4

Fase da organização	Especificação
Planejamento	É a fase em que são levantados os requisitos do negócio e os requisitos não funcionais, com base nas arguições sobre os negócios, as necessidades de clientes e usuários, e as restrições do software Também são acordados custos e prazos, efetivados após a análise
Análise	Ocorre após o levantamento de requisitos do negócio e consiste em coletar dados, definir modelos de processo de software, especificar requisitos funcionais e do sistema, determinar as atividades do desenvolvimento do software, medir o escopo do sistema e avaliar a relação necessidade/capacidade (ou custo/benefício) São elaborados cronogramas, medição de capacidade da equipe de desenvolvimento, análises de caminho crítico, modelos do processo de negócios, UC, diagrama de atividades e diagramas de componentes e implantação
Projeto	Após a viabilidade do projeto (ou design), usa-se a engenharia de software para escolher procedimentos de serviços, metodologias e técnicas apropriadas para o controle e a operacionalização do sistema que atendam aos requisitos. Definem-se as interfaces, a estrutura da informação e as topologias Criam-se diagramas de entidades e relacionamento dos dados com base em protótipos, modelos de testes (unitário de integração), classes, diagramas de sequência, diagramas de estado e outros, normalmente desenvolvidos em UML O desenvolvimento do software ocorre na sequência, quando são definidos frameworks para codificação, linguagens de programação e SGBD associados, formas de testar e diagnosticar o código
Construção	A construção está associada à implementação, quando ocorrem processos como codificação, testes, implantação do software, V&V, entrega, manutenção, treinamento e suporte técnico aos usuários Nota: a fase de desenvolvimento normalmente realiza 90% do software e os outros 10% são desenvolvidos após a execução. O fato é que os 90% previsíveis ocupam a metade do tempo total para a finalização do software, ou seja, a outra metade é ocupada na implementação de código devido a mudanças requeridas. Novos problemas não previstos surgem: correções, mudança de requisitos, melhorias no software e mudanças no ambiente operacional



Observação

Quais são as melhores técnicas e métodos da engenharia de software?

Embora todos os projetos de software devam ser desenvolvidos e gerenciados profissionalmente, diferentes técnicas são apropriadas para diferentes tipos de sistemas. Por exemplo, os jogos devem sempre ser elaborados usando uma série de protótipos, enquanto sistemas de controle críticos de segurança requerem que uma especificação completa e analisável seja desenvolvida. Não existem métodos e técnicas que sirvam para tudo (Sommerville, 2016).

4.2 Aspectos humanos da engenharia de software

Software é criado por pessoas, usado por pessoas e dá suporte à interação entre pessoas. Assim, características, comportamentos e cooperações humanas são vitais no desenvolvimento prático de software (Pressman, 2021).

São vários os profissionais que escrevem programas de software. Pessoas envolvidas na área de negócios criam programas para planilhas na automatização dos processos operacionais; cientistas e engenheiros elaboram software para processamento e análise de dados experimentais; e há pessoas que usam a programação por interesse ou para aperfeiçoar seus conhecimentos.

Contudo, a maior parte do desenvolvimento de software é feita por profissionais com a função de solucionar problemas específicos de negócios, construir algoritmos para cálculos básicos e científicos, IHC, interfaces entre dispositivos ou computadores do software, protocolos de comunicação, software para web etc.

O engenheiro de software é quem precisa dominar uma diversidade de processos, métodos, ferramentas e técnicas da engenharia de software, desenvolver habilidades para entender problemas ou necessidades para projetar soluções com qualidade.

De acordo com Pressman (2021), existem sete características pessoais que demonstram competência do engenheiro de software:

- **Responsabilidade individual:** determinação em cumprir promessas para gerência, membros da equipe e stakeholders.
- **Consciência aguçada:** ter um estado de percepção elevada, sensibilidade ao ambiente, capaz de se adaptar a emoções, mudanças, comportamentos e necessidades da equipe.
- **Extremamente honesto:** ser transparente, realista e sincero de forma construtiva para conduzir problemas e falhas.
- **Resiliência sob pressão:** a engenharia de software está sempre à beira do caos. O engenheiro de software é capaz de suportar a pressão de modo que seu desempenho não seja prejudicado (Pressman, 2021).
- **Lealdade:** à equipe e ao projeto, tendo boa vontade para colaborar e compartilhar conhecimentos, além de evitar conflitos e não sabotar o trabalho de outros. O profissional age com lealdade ao cliente e às expectativas, sempre com foco no usuário, pois seus objetivos envolvem garantir a qualidade do software entregue ao cliente e ser honesto sobre o progresso, os desafios e os possíveis atrasos no projeto.
- **Atenção aos detalhes:** não significa ser perfeccionista. O engenheiro de software deve estar atento a prazos, requisitos, desempenho, custo e qualidade estabelecidos para o projeto e o software.

- **Pragmático:** reconhece que a engenharia de software não é uma religião na qual devem ser seguidas regras dogmáticas, mas sim uma disciplina que pode ser adaptada conforme as circunstâncias (Pressman, 2021).

A estrutura da equipe e os fatores sociais governam o sucesso. A comunicação, a colaboração e a coordenação do grupo são tão importantes quanto as habilidades dos membros individuais da equipe (Pressman, 2021).

4.3 Análise de recursos para os processos do projeto

A análise de recursos para os processos do projeto é uma atividade que envolve a gerência administrativa e analistas. Essa análise envolve a identificação, a avaliação e a alocação dos recursos necessários para garantir que os processos do projeto sejam executados de maneira eficiente, eficaz e dentro das restrições, como prazo, custo e qualidade.

O PMBOK (PMI, 2021) orienta que na gerência de projetos pode haver áreas de sobreposições que ele chama de "princípios de gerenciamento de projetos" e "princípios do gerenciamento geral", conforme ilustrado na figura 32.

Praticado pela gerência administrativa, o princípio de gerenciamento de projetos inclui processos para identificar, adquirir e gerenciar os recursos necessários para a conclusão bem-sucedida do projeto, bem como comprar ou adquirir produtos, serviços ou resultados externos à equipe e subcontratações.

Por sua vez, o princípio do gerenciamento geral, praticado pela gerência da TI, refere-se ao projeto do software.

Observe com atenção. O princípio do gerenciamento geral é aplicado na produção do negócio, que pode estar ligado a diversas áreas da indústria em geral, incluindo produção de software, comércio, medicina, instituições etc. Já o princípio de gerenciamento de projetos corresponde especificamente à área administrativa ligada a fornecedores, contratos, parcerias e transporte.

Os padrões de qualidade, processos, métodos e ferramentas são diferentes entre as áreas administrativas e a de TI. No entanto, essas gerências têm algumas práticas em comum, conforme ilustrado a seguir.

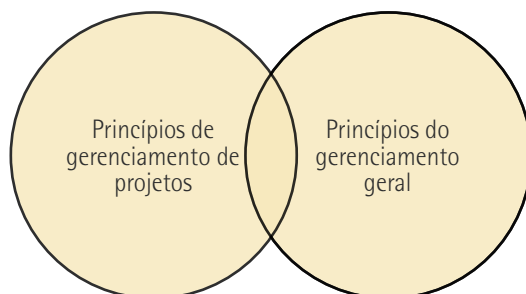


Figura 32 – Sobreposição dos princípios do gerenciamento de projetos e geral

Os objetivos da análise de recursos são:

- **Garantir as metas do projeto:** avaliar se os recursos disponíveis atendem às necessidades para atingir os objetivos do projeto.
- **Minimizar riscos:** identificar lacunas ou pontos críticos nos recursos que possam comprometer prazos, custos ou qualidade do produto.
- **Aumentar a eficiência:** alocar recursos de forma otimizada para evitar desperdícios e sobrecarga da equipe.

4.3.1 Recursos do projeto e do software

Na projeção dos recursos para atender os requisitos do software, três perfis de analistas são essenciais:

- **Analista de negócios (ou autor do processo):** responsável por identificar as necessidades do negócio, sejam elas de geração, correção ou melhoria; visando atender objetivos ou necessidades específicas. Ele utiliza uma linguagem natural e diagramas simples para comunicar claramente o que é exigido e o que precisa ser feito.
- **Analista de processos:** interpreta a ideia do negócio, avaliando seus riscos e suas regras, com o objetivo de determinar as atividades e as tarefas necessárias para processar o negócio. Ele busca padronizar a nomenclatura e especificar, por meio de textos concisos e diagramas detalhados, a ordenação das atividades e das tarefas que compõem o processo do negócio.
- **Analista de sistemas:** converte as atividades em componentes específicos que integram o processo. Cada componente representa uma parte do software suportado pelo processamento de dados. A interpretação das tarefas e do workflow (fluxo de trabalho) leva à criação das funções que irão operacionalizar o software, a lógica de processamento (programa de computador) e os recursos computacionais necessários para executar essas funções.

Na prática da engenharia de software, o analista de sistemas especifica cada função através de descrições detalhadas e modelos gráficos (modelagem), determinando as funcionalidades que serão interpretadas pelos especialistas em TI.

Com base nos requisitos definidos pelos analistas de negócios, do processo e de sistemas, para desenvolver o software serão necessárias duas gerências: a de projeto e a de TI. Vamos estudá-las a seguir.

Gerência do projeto

Promove o projeto em todos os aspectos, o que significa apoiar, defender e divulgá-lo na organização tanto interna como externamente. Essas ações ajudam a criar um ambiente positivo e de suporte ao redor do projeto, aumentando as chances de sucesso e de adesão tanto dos stakeholders quanto da equipe de desenvolvimento.

Um stakeholder é qualquer pessoa, grupo ou organização que tem interesse ou é afetado pelo resultado de um projeto, produto ou negócio; envolve as partes interessadas que podem influenciar ou serem influenciadas pelo que está sendo desenvolvido ou implementado.

O gerente de projeto:

- define os limites e os objetivos do projeto;
- mantém uma comunicação clara e contínua com stakeholders e a equipe de desenvolvimento;
- desenvolve, analisa e entende os processos envolvidos conforme os requisitos;
- aloca tarefas e responsabilidades de acordo com as habilidades e competências;
- estabelece uma estrutura organizada para administrar e monitorar o progresso da equipe;
- estima e controla os custos do projeto de acordo com o orçamento.

Os principais recursos sob a gerência do projeto incluem:

- **Recursos humanos:** os recursos humanos envolvidos em um projeto variam de acordo com o tamanho, a complexidade e os objetivos do projeto. Esses setores atuam no suporte, na coordenação e na governança, garantindo que o projeto seja conduzido de forma eficiente e alinhada aos objetivos organizacionais. Incluem todos os envolvidos e interessados, os coordenadores (que auxiliam na comunicação entre equipes), os analistas etc. Acentua-se, por fim, que os recursos consumidos pela gerência de TI também estão sob a gerência do projeto.
- **Recursos materiais:** referem-se a localização, transporte, mobília e instalações para o ambiente de uso e ambiente de desenvolvimento do software. O objetivo é criar um ambiente de trabalho eficiente e confortável, que suporte as atividades do desenvolvimento e os testes de software. A gerência de TI é responsável por fornecer a lista de recursos tangíveis consumidos pela equipe de desenvolvimento.
- **Recursos tecnológicos:** envolvem os recursos intangíveis fornecidos pela gerência de TI, como ferramentas de software, licenças, plataformas de desenvolvimento e infraestrutura de TI. Esses recursos são essenciais para o desenvolvimento de software, garantindo que a equipe tenha as tecnologias necessárias para atender às expectativas dos usuários em termos de funcionalidade e desempenho.
- **Recursos financeiros:** fornecem o financiamento necessário para cobrir todos os custos associados ao desenvolvimento do software, incluindo salários, ferramentas, infraestrutura e outros recursos. A gestão eficiente dos recursos financeiros é crucial para garantir que o projeto seja concluído dentro do orçamento e atenda às expectativas dos stakeholders e usuários finais.

Gerência da TI

Ela envolve a administração dos recursos tecnológicos necessários para construir ou produzir software.

Criar ou produzir software pode estar associado ao seu desenvolvimento por completo: módulo, componente ou subproduto dele. O gerenciamento da TI corresponde às fases do ciclo de vida do desenvolvimento de software, que incluem áreas específicas da aplicação.

No ambiente computacional informatizado, o desenvolvimento do software abrange a administração da infraestrutura de TI, que suporta o desenvolvimento de sistemas em uma sequência ordenada que acompanha esse ciclo. Os principais recursos tecnológicos necessários para desenvolver software incluem:

- **Recursos de software:** referem-se a ferramentas, plataformas e soluções digitais fáceis para desenvolvimento, gestão, implantação e manutenção de sistemas e aplicativos. Esses recursos são classificados como intangíveis e desempenham um papel essencial em todas as etapas do ciclo de vida do desenvolvimento de software. Os principais recursos de software são:
 - Ferramentas de desenvolvimento, a exemplo de IDEs como Visual Studio, Eclipse, Netbeans e PyCharm. São ferramentas que oferecem funcionalidades como edição de código, depuração e execução. O engenheiro de software tem ao seu dispor inúmeros outros dispositivos que dão apoio ao desenvolvimento: aparelhos de modelagem e arquitetura, ferramentas CASE (Computer-Aided Software Engineering – Engenharia Assistida por Computador), ferramentas de gerenciamento de projetos e muitas outras.
 - Editores de código como VS Code, Sublime Text ou Notepad++.
 - Compiladores e interpretadores para linguagens de programação, o software traduz o código-fonte para o código correspondente ao da máquina, por exemplo: GCC (GNU Compiler Collection – Coleção de Compiladores GNU) e JDK (Java Development Kit – Kit de Desenvolvimento Java).
 - Licenças de software ou utilitários para manutenção do sistema no ambiente do desenvolvedor, tais como sistemas operacionais, aplicativos e gerenciadores de banco de dados. O software livre que estiver útil no desenvolvimento deve compor essa lista.
- **Recursos de hardware:** são recursos tangíveis que garantem o desempenho necessário para executar o software com bom desempenho, oferece suporte para o desenvolvimento de sistemas escaláveis e seguros e que possibilitam conexões e operações em ambientes corporativos. Os principais recursos de hardware são:
 - Computadores servidores e estações de trabalho usadas por desenvolvedores, analistas, programadores, gerenciadores do banco de dados e testadores. São considerados como estações de trabalho: desktops, notebooks e laptops.

- Dispositivos de armazenamento para o ambiente de desenvolvimento e para infraestrutura de serviços, bem como dispositivos de backups.
 - Periféricos, componentes como teclado, mouse, monitores e impressoras.
 - Acessórios, dispositivos de entrada como webcam, tablets ou scanner.
 - Infraestrutura de rede, tais como placas de rede, roteadores, modem, switches, pontos de acesso, cabos, firewalls e dispositivos de segurança.
 - Infraestrutura física e de suporte como racks, ar-condicionado, nobreak, câmeras de segurança, sensores de movimento e dispositivos inteligentes.
- **Recursos humanos (peopleware):** são reservados aos especialistas que trabalham no desenvolvimento do software. Esses profissionais têm diferentes responsabilidades e especializações. Os principais peopleware são:
 - Gestores e líderes, como o gerente do projeto de software, que coordena todas as atividades do projeto, gerencia recursos, cronogramas e riscos e mantém a comunicação com os stakeholders. Também são citados o product owner, que representa os interesses dos stakeholders e dos usuários, e o Scrum master, que facilita o processo ágil, removendo obstáculos e garantindo que a equipe siga as práticas Scrum.
 - Programadores: escrevem o código, implementam funcionalidades e resolvem problemas técnicos. Acentua-se que o front-end tem como foco a IU. Já a UX usa tecnologias como HTML, CSS e JavaScript. Por sua vez, os back-end trabalham na lógica do servidor, dos bancos de dados e da integração de sistemas e utilizam tecnologias como Java, Python, Ruby e outras.
 - IU e UX: são os especialistas em usabilidade. Eles analisam o uso do software e fazem recomendações de melhorias. O designer de UI/UX cria visuais atraentes e intuitivos para melhorar a experiência.
 - Equipe de suporte e manutenção.
 - Outros profissionais que atuam no desenvolvimento de software, como o engenheiro de software, responsável pela criação de soluções tecnológicas que atendem a necessidades complexas e, ao mesmo tempo, escaláveis e sustentáveis, acompanhando todo o ciclo de vida do software. Citam-se ainda engenheiros de DevOps, responsáveis por integrar desenvolvimento e operações para automatizar processos de construção, teste e implantação; e os especialistas em segurança, que garantem que o software seja seguro contra ameaças e vulnerabilidades.
 - **Recursos de dados:** envolvem qualquer tipo de informação ou dados que podem ser usados para análises, tomada de decisões ou alimentar sistemas de software. Permitem manipular a

informação de forma eficiente e garantem a entrega de soluções que atendam às demandas do negócio e dos usuários. Os principais recursos de dados são:

- **Dados coletados diretamente da fonte:** dados estruturados como tabelas, dados não estruturados como imagens, vídeos, documentos e demais extensões de arquivos.
- **Base de dados (databases):** conjuntos organizados de dados armazenados e acessados eletronicamente. Incluem bancos de dados relacionais (SQL): MySQL, Microsoft SQL, PostgreSQL e Oracle; não relacionais (NoSQL): MongoDB e Cassandra; em nuvem: Azure Cosmos DB e AWS RDS.
- **APIs (Application Programming Interfaces – Interface de Programação de Aplicações):** englobam um repositório de rotinas e ferramentas para construir o software, permitindo acesso e interação com dados de outros sistemas.
- **Streams de dados:** são fluxos contínuos de dados em tempo real, como logs de servidor, dados de sensores, redes sociais e plataformas de vídeo.
- **Data Warehouse e Big Data:** repositórios de grande volume de dados e múltiplas fontes que fornecem como resultados relatórios analíticos para tomada de decisões.
- **Segurança e proteção de dados:** como criptografia, controle de acesso, backups, monitoramento e auditoria.
- **Recursos da rede de computadores:** são serviços e componentes associados que permitem uma conectividade com bom desempenho, segura e escalabilidade. O investimento em recursos de rede traz um bom funcionamento de qualquer infraestrutura de tecnologia da informação, e os principais deles são:
 - **Recursos físicos:** componentes tangíveis, responsáveis pela conectividade e pela infraestrutura de rede. Eles devem compor a lista dos recursos de hardware e suas configurações fazem parte dos recursos da rede de computadores. Os principais componentes são: roteadores, switches, access points (pontos de acesso de dispositivos Wi-Fi em redes sem fio), modems, servidores de rede, periféricos associados, cabos e conexões.
 - **Infraestrutura de comunicação:** meios físicos e lógicos que suportam a troca de dados como topologias de rede, mapas de endereçamento, backbones e conectividade de nuvem.
 - **Recursos lógicos:** protocolos de rede como TCP (Transmission Control Protocol – Protocolo de Controle de Transmissão), HTTP/HTTPS, FTP, DNS e SMTP/IMAP/POP3. Citam-se também sistemas operacionais de rede (SOR) como Windows Server, UNIX e Linux (as licenças dos sistemas operacionais fazem parte dos recursos de software).

- **Armazenamento de rede (network storage):** dispositivos ou sistemas de armazenamento acessíveis através da rede, como NAS (Network Attached Storage – Armazenamento Conectado à Rede) e SAN (Storage Area Network – Rede de Área de Armazenamento).
- **Recursos de segurança e controle:** ferramentas e práticas para proteger a integridade e a confidencialidade da rede, como firewall, VPN, criptografia de dados, antivírus, IPS (Intrusion Prevention System – Sistema de Prevenção de Intrusão), controle de acesso e monitoramento da rede.

4.4 Métodos, ferramentas e técnicas: modelagem da infraestrutura de TI

No projeto de software, basicamente são desenvolvidas três arquiteturas: lógica de processamento, informação e infraestrutura de TI. Elas foram abordadas nas atividades de modelagem, comentadas quando estudamos o Cascata.

As arquiteturas da lógica de processamento, da informação e da infraestrutura de TI, assim como diversos modelos derivados, compõem um repertório de alternativas de componentes para construir sistemas de software que ficam armazenados em um repositório e disponíveis para os desenvolvedores.



Observação

O repositório do desenvolvimento de sistemas de software é um local de armazenamento de "peças" de software para sua construção.

O repositório do desenvolvimento de sistemas (figura 33) não apenas armazena dados e informações, mas é útil para a gestão e o compartilhamento de vários elementos do projeto de software, como componentes, módulos, objetos, projetos, modelos, ferramentas de software, documentos ou quaisquer outros artefatos produzidos para implementar ou construir sistemas de software.

Normalmente, as ferramentas de desenvolvimento da indústria de software de qualquer tamanho trabalham com seu repertório de alternativas para o projeto de software. Elas ficam armazenadas em repositórios, que podem estar alocados na estação de trabalho, como é o caso da aplicação Astah Community, ou alocados em servidor ou nuvem, a exemplo do RUP.

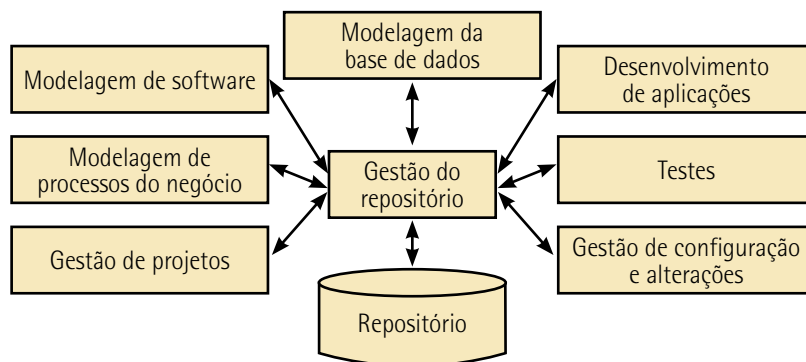


Figura 33 – Arquitetura de ferramentas de software com repositório para desenvolvimento de sistemas de software

A reusabilidade de componentes de software é um princípio da engenharia de software, uma métrica da qualidade usada para avaliar o quanto um programa ou parte dele pode ser usada em outras aplicações. Tem como foco a criação de módulos de código que podem ser utilizados em diferentes aplicações, sem a necessidade de retrabalho significativo. A figura 34 mostra o conteúdo de um repositório com um repertório de alternativas de componentes de software.

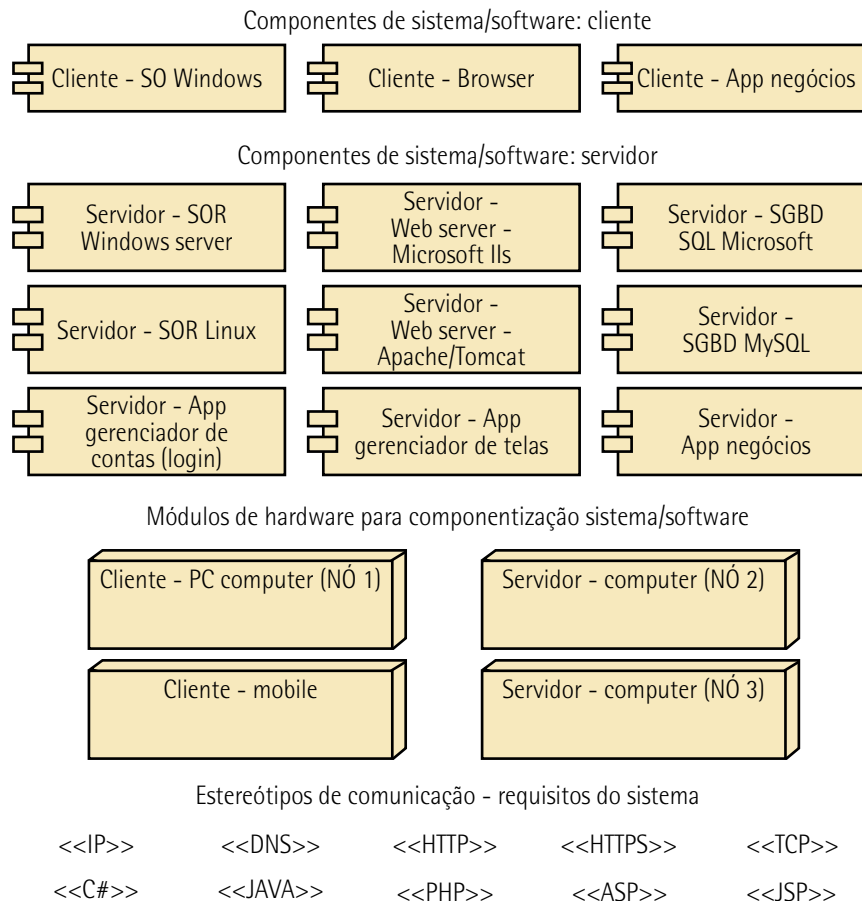


Figura 34 – Repertório de alternativas de componentes e módulos para a construção de uma infraestrutura de TI para diversos sistemas

O componente de software é uma unidade modular e independente de código externo, ou seja, tem endereçamento próprio e os dados são consumidos no componente, o que significa que é possível fazer pequenas mudanças nele sem afetar a integridade do sistema. O componente realiza uma função específica dentro de um sistema maior. Assim, esses componentes podem ser desenvolvidos de maneira isolada e, depois, integrados a outros para construir aplicações.

Conforme a figura 35, no padrão UML, o bloco componente pode ter duas formas, sendo representado por uma caixa e dois tabs:

- na caixa são descritos o nome do componente e do(s) estereótipo(s), que classificam o componente;
- os tabs são as interfaces de entrada/saídas responsáveis pela ligação com outros componentes.

Ainda na figura 35, vemos os blocos de implantação, que representam os módulos, em que estão encapsulados diversos componentes. O bloco de implantação (UML) é apresentado como uma caixa em 3D com deslocamento da aba para cima e à direita. Um módulo pode estar conectado a outros módulos ou componentes. Logo mais abaixo da figura 35 estão os estereótipos, usados como um mecanismo de extensão, que permite adicionar novos elementos ou propriedades aos elementos da UML. Eles personalizam o elemento UML (apresentado no formato <<nome do estereótipo>>) para se ajustar às necessidades dos requisitos.

A modelagem da infraestrutura de TI ajuda a alinhar a infraestrutura de TI com os objetivos e as necessidades da organização, garantindo que os recursos tecnológicos sejam implementados. Ela permite otimizar recursos de hardware e software e melhora a segurança com a identificação de pontos críticos, trazendo uma visão escalável e resiliente da arquitetura que garante a continuidade das operações, mesmo diante de falhas ou incidentes, permitindo a implementação de novas tecnologias.

Já a modelagem da arquitetura do software é construída a partir dos requisitos, principalmente os RS. Sua construção ocorre pelo processo de componentização do software.

A componentização é uma abordagem de design e desenvolvimento na qual o sistema ou aplicação é dividido em componentes independentes e reutilizáveis. Cada componente é uma parte modular do sistema com uma função específica.

Na UML, um nó em um diagrama de implantação representa um recurso físico como dispositivo de hardware, servidor ou estação de trabalho, ambiente de execução de software como uma máquina virtual ou um contêiner, ou um recurso lógico que pode executar artefatos (software, bibliotecas ou arquivos).

A modelagem da infraestrutura de TI pode ser elaborada com o software Astah Community ou simplesmente Astah, ilustrado na figura 35. Trata-se de uma ferramenta de modelagem e diagramação usada por desenvolvedores de software, engenheiros e analistas de negócios. Criada pela Change Vision, Inc., a Astah é conhecida por oferecer soluções que facilitam a visualização e o design de sistemas complexos.

Potencializando a excelência em engenharia com a Astah

As ferramentas de modelagem da Astah permitem que você visualize a essência de suas ideias e designs de software. Crie diagramas de forma rápida e sem esforço que criem um entendimento claro entre as equipes.

Crie UML, diagramas ER, diagramas de fluxo de dados, fluxogramas, mapas mentais e muito mais no software de modelagem mais poderoso para todos, desde estudantes até equipes corporativas (Changevision, 2006).

Observe a seguir algumas das principais características do Astah:

- **Modelagem UML:** tem uma ampla gama de ferramentas para criar diagramas UML, incluindo de classes, de UC, de sequência, de atividades etc.
- **Diagramas ER e DFD:** além dos diagramas UML, o Astah permite a criação de diagramas de entidade-relacionamento (ER) e de fluxo de dados (DFD), úteis para modelar dados e processos de negócios.
- **Interface intuitiva:** com recursos de arrastar e soltar que facilitam a rápida criação de diagramas.
- **Colaboração em equipe:** há ferramentas que permitem que equipes trabalhem juntas em projetos, compartilhando e mesclando diagramas facilmente.
- **Exportação e importação:** os diagramas criados no Astah podem ser exportados em vários formatos, como imagens (JPG, PNG, EMF, SVG), documentos RTF, HTML, relatório de definição de entidade e XML.
- **Versatilidade:** o software é usado em diversas indústrias, como TI, desenvolvimento de software e engenharia de sistemas, para criar representações visuais claras e padronizadas de sistemas complexos.

Exemplo de aplicação

A arquitetura da infraestrutura de TI apresentada na figura 35 tem quatro nós: CLIENTE 1 – Estação PC, CLIENTE 2 – Estação Mobile, SERVIDOR 1 – SOR e SERVIDOR 2 – APPs, SGBD. Suas ligações estão em rede e os meios de comunicação estão assinalados pelos estereótipos de protocolos <<TCP>>, <<IP>>, <<HTTP>> e <<DNS>>.

Analisando a arquitetura, vemos que é uma arquitetura servidor/cliente. CLIENTE 1 é um computador de trabalho padrão (desktop ou notebook), CLIENTE 2 usada é para acesso remoto de usuários, SERVIDOR 1 é o servidor que faz frente ao usuário e o SERVIDOR 2 é onde estão localizados os serviços das aplicações e dos bancos de dados.

O Astah tem um repositório que permite compartilhar os artefatos de software produzidos com a equipe de desenvolvimento. Vários componentes vão desde SOR, apps, SGBD e aplicações de servidores, como tratamento de tela, contas do usuário e relatórios.

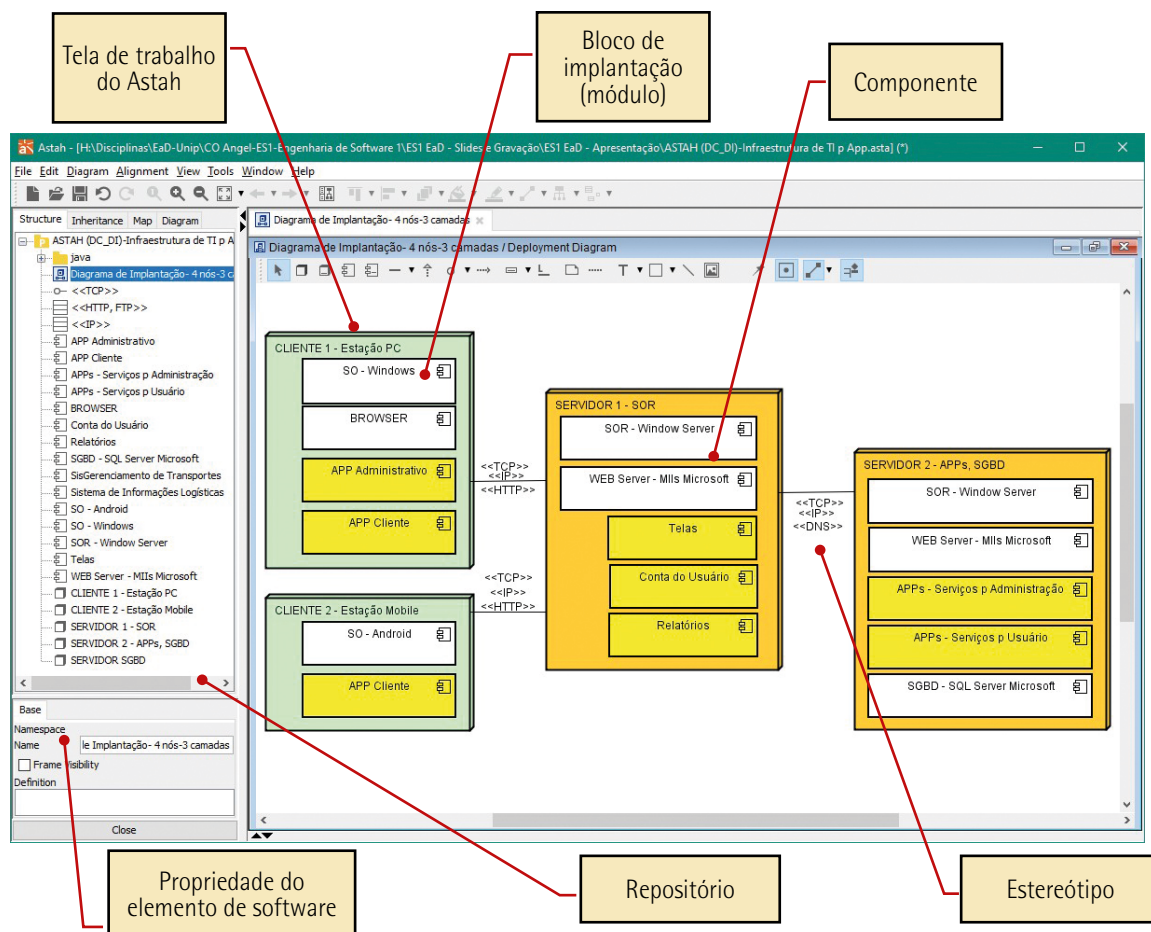


Figura 35 – Ferramenta de desenvolvimento Astah Community com a modelagem da arquitetura de infraestrutura de TI



Saiba mais

Explore mais a componentização com a IBM:

IBM. *Rational software architect standard edition*. [s.d.]. Disponível em: <https://shre.ink/egum>. Acesso em: 23 jun. 2025.

Explore suas ideias de design de software e coloque isso em prática. Faça download do software Astah e pratique suas funcionalidades.

Disponível em: <https://shre.ink/egKy>. Acesso em: 23 jun. 2025.

Consulte também:

ASTAH. *Powering engineering excellence with Astah*. [s.d.]. Disponível em: <https://shre.ink/egKD>. Acesso em: 23 jun. 2025.



Resumo

Abordamos nesta unidade o processo de software, determinando que todo o processo é composto por um conjunto de atividades e tarefas que, organizadas, conduzem a uma prática ou estratégia que melhoram a produtividade.

Vimos que é essencial que todo desenvolvedor ou equipe tenha sob domínio um arcabouço do processo com os recursos necessários para o projeto e a construção do produto, algo que contemple a agilidade no desenvolvimento de software e as iterações do processo para obter capacidade de respostas rápidas às mudanças que ocorrem ao longo do ciclo de vida do desenvolvimento. Nesse contexto, foram destacados os modelos prescritivos, o desenvolvimento ágil e a subdivisão em subprocessos, atividades, procedimentos e tarefas. Os principais processos foram: PSP, TSP, Cascata, Balbúrdia, Prototipagem, Incremental, RAD e Espiral.

Em seguida, detalhamos o planejamento do processo de software relacionando-o ao SDLC. Enfatizamos a estrutura organizacional como regra para produzir um bom software com métodos, ferramentas e técnicas específicas. Vimos a importância de montar um cronograma para planejar, monitorar e controlar os recursos, pois a inteligência organizacional é responsável pelo sucesso do software.

O planejamento do processo de software compõe a integração da estrutura organizacional alinhada com os recursos para os processos do projeto subdivididos nos gerenciamentos de projeto. Dessa forma, na projeção dos recursos são conceituados os perfis profissionais da análise do negócio, do processo e do sistema, avaliando-se os recursos sob a gerência do projeto e sob a gerência da TI.

Por fim, acentuamos os MF&T aplicados à modelagem da infraestrutura de TI, com destaque para o repositório do sistema, a reusabilidade, o componente, o módulo e os estereótipos.



Exercícios

Questão 1. (Fadesp 2024, adaptada) Sobre o modelo de desenvolvimento de software Espiral, avalie as afirmativas.

I – Uma das características mais marcantes do Espiral é a sua ênfase na identificação, análise e mitigação de riscos.

II – Segue a abordagem de passos sistemáticos do Cascata, incorporando-os a uma estrutura iterativa.

III – É uma abordagem realista para o desenvolvimento de sistemas de software de grande porte.

É correto o que se afirma em:

A) I, apenas.

B) II, apenas.

C) I e II, apenas.

D) II e III, apenas.

E) I, II e III.

Resposta correta: alternativa E.

Análise das afirmativas

I – Afirmativa correta.

Justificativa: uma das principais características do Espiral é justamente o gerenciamento de riscos, que ocorre em cada iteração do processo. Ele identifica, analisa e mitiga riscos sistematicamente.

II – Afirmativa correta.

Justificativa: o Espiral combina elementos do Cascata (abordagem linear) com iterações (ciclos repetitivos). Cada ciclo passa por fases como planejamento, análise de riscos, desenvolvimento e avaliação, incorporando aspectos sequenciais e evolutivos.

III – Afirmativa correta.

Justificativa: o Espiral é especialmente adequado para sistemas complexos e de grande porte, pois permite ajustes contínuos, mitigação de riscos e evolução incremental, sendo mais conveniente e realista do que modelos puramente lineares, como o Cascata.

Questão 2. (FGV 2024, adaptada) A analista Luísa foi designada como responsável pelo acompanhamento de todo o ciclo de vida de um software.

Sobre o ciclo de vida do produto, é correto afirmar que:

- A) Finaliza com a entrega do produto.
- B) É similar ao ciclo de vida de um projeto qualquer.
- C) Apresenta escopo bem definido antecipadamente.
- D) O seu sucesso é medido exclusivamente por prazos e orçamento.
- E) É similar ao ciclo de vida de programas de longa duração.

Resposta correta: alternativa E.

Análise das alternativas

A) Alternativa incorreta.

Justificativa: o ciclo de vida de um software vai além da entrega inicial, incluindo manutenção, atualizações e eventual descontinuação.

B) Alternativa incorreta.

Justificativa: projetos simples e de requisitos estáveis têm início, meio e fim definidos; enquanto produtos de software têm um ciclo contínuo e com evolução constante.

C) Alternativa incorreta.

Justificativa: principalmente em metodologias ágeis, o escopo pode evoluir durante o desenvolvimento.

D) Alternativa incorreta.

Justificativa: embora prazos e orçamento sejam importantes, o sucesso também depende da qualidade, da satisfação do usuário e de outros fatores.

E) Alternativa correta.

Justificativa: assim como programas de longa duração, produtos de software têm um ciclo contínuo com fases de desenvolvimento, operação e manutenção. Isso é evidente em softwares que, após serem lançados, precisam de atualizações constantes e de suporte.